Data and Decision Traceability for SDA TAP Lab's Prototype Battle Management System

Yasir Latif, Latha Pratti, Samya Bagchi - Space Protocol LLC,
{yasir,latha,samb}@spaceprotocol.org

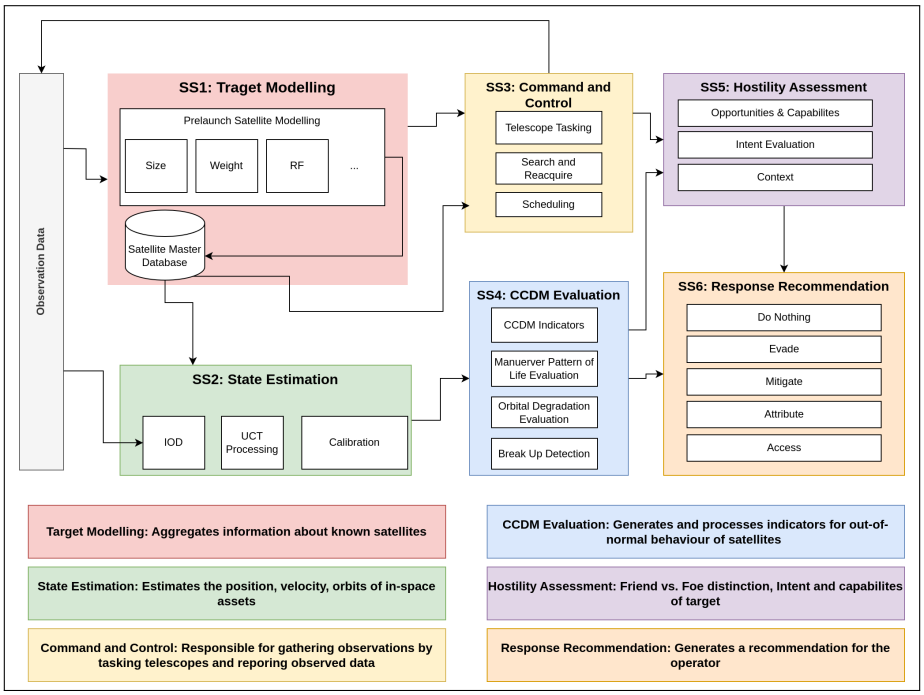

Figure 1. The Prototype Battle Management System grouped into its functional subsystems.

1 Introduction

A complex battle management system (BMS) for operator recommendation is being developed for the SDA TAP Lab that ingests sensor input, historic data and other information on Space Domain Awareness (SDA), conducts hostility assessment of Resident Space Objects (RSOs), and provides response recommendations for operator use. The current design of BMS (see Figure. 1) consists of seven subsystems with complex data interactions between them, however, it provides opaque recommendations: the operator only sees the response recommendation and is unaware of the complex sequence of decisions made at each step by a different algorithm that lead to the recommendation. This might be acceptable behavior if the system is guaranteed to perform optimally; however, in cases where an intermediate error

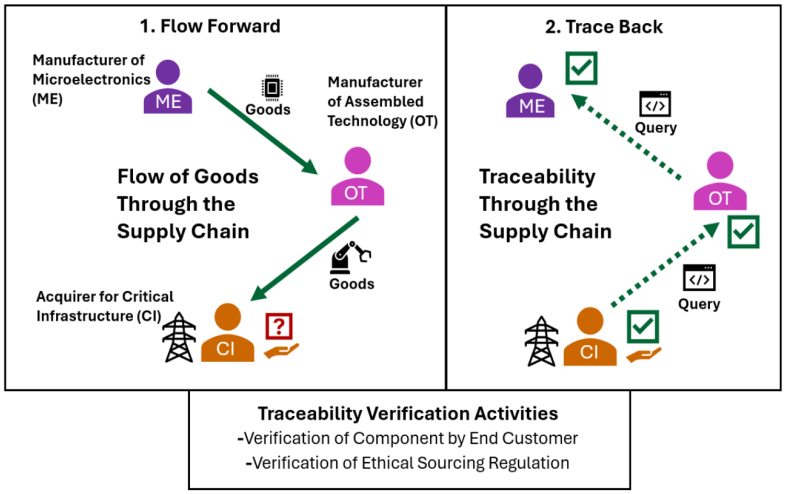

Figure 2. Flow of goods (forward through time) and traceability (backwards through time): Use case of information verification within a manufacturing supply chain. (Figure from [1])

in the pipeline leads to an incorrect recommendation, detailed auditing of the system, replay of the sequence of events, and fine-grained introspection are required to isolate the source of error within this complex system.

2 Supply Chain Traceability and the Target Model Database (TMDB)

Space Protocol has been actively involved with MITRE Corporation since May 2024 and has been collaborating on the implementation of their NIST IR 8536 "Supply Chain Traceability: Manufacturing Meta-Framework" [1]. The comprehensive framework is designed to enhance traceability across manufacturing supply chains, focusing on improving product provenance, pedigree, and supply chain transparency. The Meta-Framework organizes, links, and enables querying of traceability data across manufacturing supply chains (Figure. 2).

Within the realm of the SDA TAB Lab, this framework is directly applicable to the Target Model Database (TMDB), which encompasses comprehensive details regarding space assets. Our initial proposal was to investigate the feasibility of establishing a manufacturing supply chain for satellites that would integrate with the TMDB. Nevertheless, the acquisition of such data from public sources quickly emerged as an insurmountable obstacle in this endeavor and, we subsequently explored applying the same core principles of the NIST framework to data and decision traceability within the Welder's Arc system.

3 Decision Traceability for BMS

Space Protocol is applying the principles derived from [1] to the the BMS to achieve introspection, auditing, and replay of data and decisions that ultimately lead to a response recommendation. The core goal of **decision traceability** is to ensure transparency, accountability, and integrity within the BMS system. This is accomplished by providing a clear, auditable path from the system's inputs all the way to the final decision. This traceability enables the system to track the various algorithms and data flows that have influenced a particular outcome.

The incorporation of the message bus as the common communication framework, as opposed to the earlier model of end-to-end communication between different algorithms via API calls, has opened up the possibility of end-to-end data and decisions traceability within the system. Instead of direct API calls, different algorithms now send messages to the message bus to make their contributions known. Each message consists of a well-defined schema, that captures key information about data, its origin and the process that generates it (see Listing. 1 for an example). Using the message bus as the common communication framework enables every component of the system to contribute details about its inputs, outputs, and metadata with each interaction. This data is collected as the system evolves, providing a chronological log of all the data transformations and decisions over time.

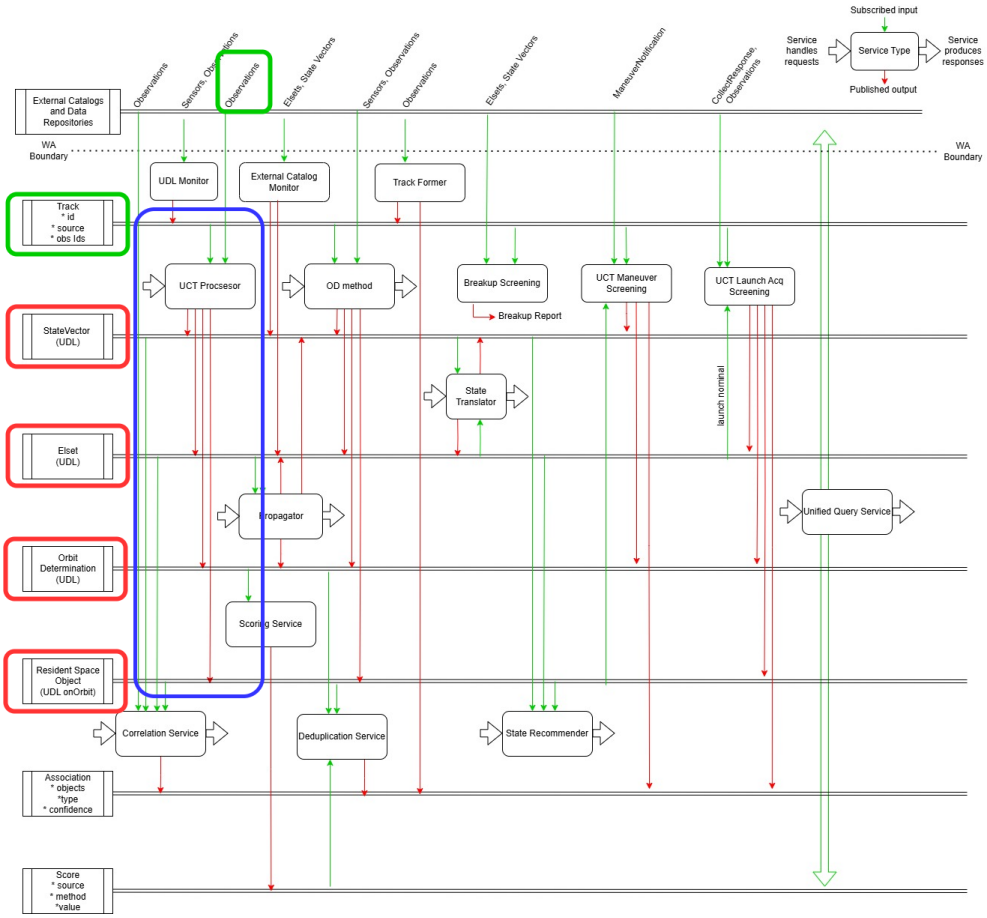

Figure 3. Data required for traceability: Inputs (green) and outputs (red) of an algorithm are captured via the header schema along with metadata for a UCT processing algorithm (blue box).

The message bus serves as the central mechanism for collecting the data needed for traceability. It is being developed through a collaboration between Millennial Software and Pacific Northwest National Labs (PNNL), with Space Protocol contributing to the design of the message headers that enable the traceability. This collaboration ensures the framework supports all components within the BMS system.

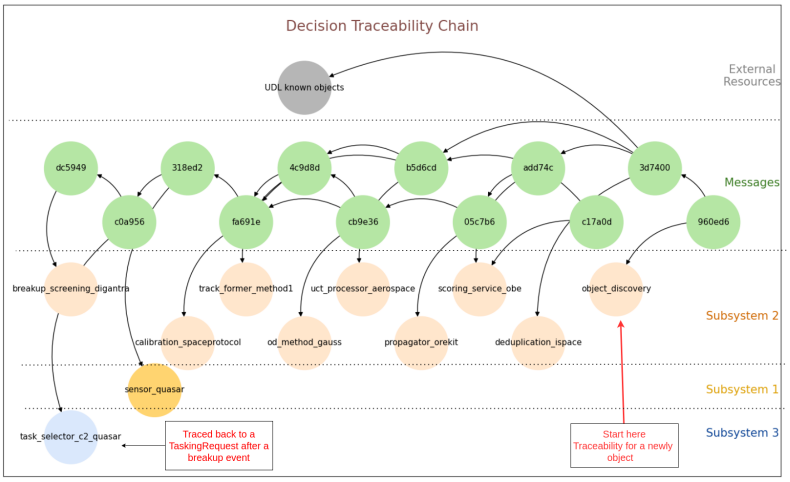

Figure 4. Traceability links a newly discovered object to the breakup event in which it originated.

An example required for traceability is shown in Figure 3 where a UCT processing algorithm consumes `tracks` and `observations` and generates four distinct outputs. The underlying message bus schema enables collection of metadata about each unique input and output of the algorithms. When needed, any decision (output) can be traced back to its immediate parents, which in turn are related to their parents, building up the complete traceability chain that provides data and decision visibility.

3.1 Traceability: Object Discovery

To illustrate the practical application and value of the traceability framework, the scenario of a new object being discovered following a satellite breakup event is considered. This scenario exemplifies how the system traces back from the identified new object to the initial events that led to its discovery. The sequence of algorithms and data flows involved in this object discovery process is shown in Figure 7 (Appendix A), and the corresponding decision traceability chain is presented in Figure 4.

The traceability framework, inspired by MITRE's NIST IR 8536 framework, is implemented by using a message bus, that ensures that every interaction between algorithms is captured as a message. This is an important component that facilitates backwards traces.

- **Message Schema**: Each message contains metadata about the data, including the unique ID of the message, the parent message ID(s), and algorithm that produced the data. This metadata is the key in connecting the messages to each other for traceability.
- **Parent Relationships**: Messages contain links to their parent messages and external data sources, allowing the system to retrace the entire chain of events backwards to the source.

Figure 4 shows the result of this trace back process, where a newly discovered object can be traced back through a number of messages and sub-system components (represented as orange, yellow, and blue circles) to the original task request that triggered the breakup investigation. The green circles represent the messages, which are the results of various processing steps performed by the system. The external data sources consumed by the processing steps are represented as gray circles. By showing the entire process, in reverse order, the framework

provides complete visibility of the data and processes that went into the decision. Decision traceability for this object discovery scenario provides the following benefits:

- **Auditing**: Enables system engineers to audit the decision paths backwards, verifying that the system made the correct decision by reviewing the input data and the various models that contributed to the final decision.
- **Error Diagnosis**: If a questionable or incorrect object is discovered, operators can use the chain to quickly trace back to the point at which the problem occurred; for instance, a model may have generated a questionable output, or the wrong data was used.
- **Continuous Improvement**: The insight gained from the traceable data and processes allows for continuous improvement by refining the models and the data, and by understanding the impact on the decision, which will ultimately result in more reliable outcomes in the future.

4 Next Steps

To implement data and decision traceability, the following steps are being pursued:

- **Integration with the message bus**: A key objective of this work is to establish comprehensive traceability within the BMS system. The immediate next step is the onboarding of all algorithms the message bus, which will enable the automatic capture of data related to their inputs and outputs. Given our existing system integration with the message bus, this data capture will begin as soon as each algorithm is integrated. This data will not only enable post-hoc analysis of system behavior but also provide valuable insights for future system optimization and development.
- **Graph Database (DB)**: To facilitate comprehensive traceability and auditing capabilities within the BMS system, metadata derived from the message bus will be stored in a graph database (DB). Graph databases are particularly well-suited for this purpose due to their ability to efficiently represent and manage complex relationships. By storing metadata as a graph, we can effectively reconstruct the flow of information and the sequence of processing steps that lead to specific outcomes.
- **Traceability User Interface (UI)**: To facilitate user access to the traceability data, a User Interface (UI) is being developed (Fig. **??** that will allow users to select a specific decision or event and visualize its corresponding traceability chain. The UI employs graphical representations, such as directed graphs or flow diagrams, to effectively convey the complex relationships between data elements and processes. This visualization enables users to quickly grasp the sequence of events and identify critical dependencies, facilitating efficient analysis and understanding of system behavior.

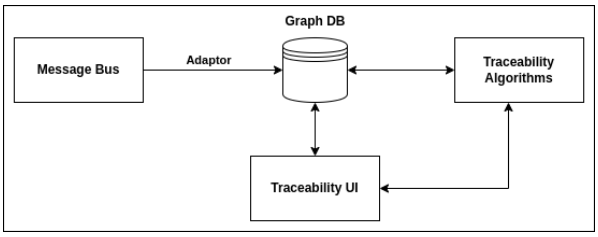

Figure 5. System architecture of the Traceability framework

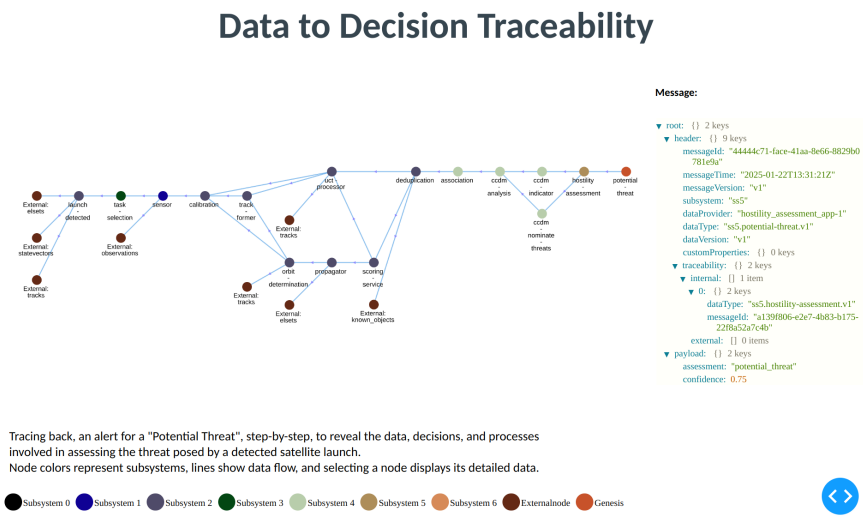

Figure 6. Proposed User Interface (UI) for data and decision traceability capable of demonstrating the algorithms and messages ingested.

References

[1] M. Pease, F. Wallace, H. Reed, V. Martin, S. Granata, *Supply Chain Traceability: Manufacturing Meta-Framework*, in *NIST Internal Report* (National Institute of Standards and Technology (NIST), 2024), NIST IR 8536, Initial Public Draft, `https://csrc.nist.gov/pubs/ir/8536/ipd`

A Reference Algorithms and Data Flow

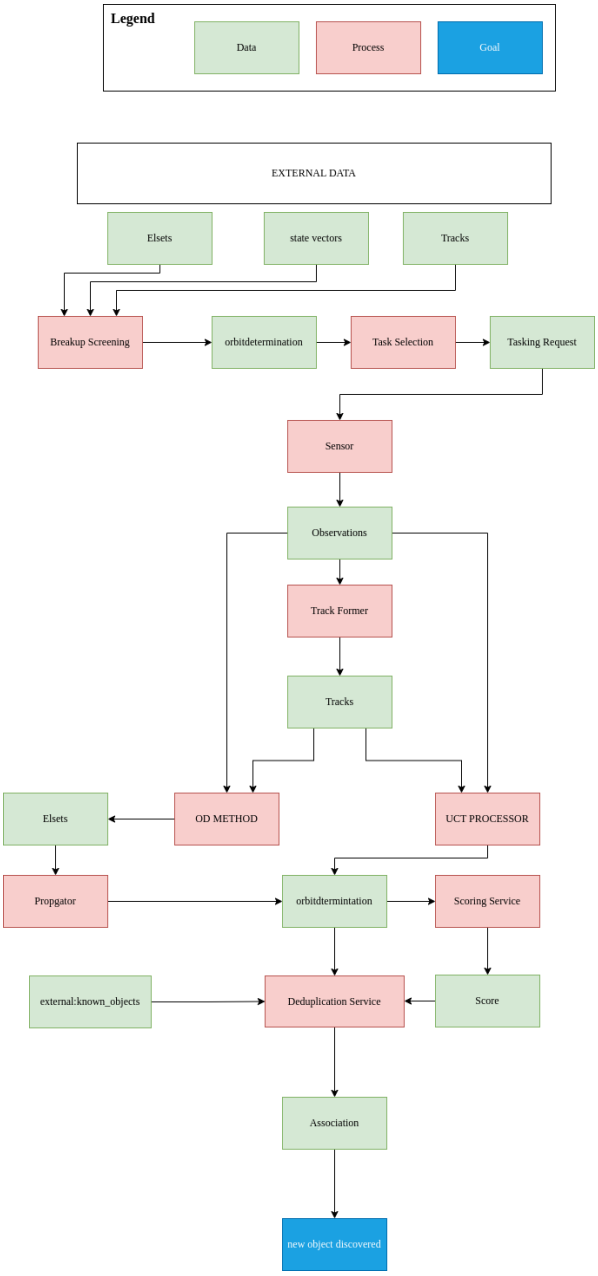

Figure 7. A small scale data and algorithm flow that approximates the message bus.

Figure 7 presents a simplified data and algorithm flow that illustrates how the message bus functions as the central communication mechanism within the Welder's Arc system. In this diagram, 'Data' boxes represent specific data elements, such as tracks and observations. 'Process' boxes represent computations or algorithms, like orbit determination and breakup

screening. And ‘Goal’ boxes represent a specific outcome in the process, such as discovering a new object. All interactions between these elements – data transfer, algorithm invocation, and state changes – occur through the message bus, enabling the system’s data traceability.

B Message Schema

Listing 1 presents an example of a message header incorporating the traceability section. To facilitate comprehensive tracking of data provenance, parent relationships are categorized as either `internal` or `external`. Internal parent relationships denote data originating within the BMS system via the message bus, while external relationships represent data sources outside WA’s boundaries, such as API calls and bulk data requests. For each identified parent, the message header records the data type and the unique message ID. Additionally, a designated `parameters` field is included to accommodate supplementary information for external resources, such as API call parameters or other relevant metadata.

Listing 1. Example Header for Calibration Message

```
{
  "messageId": "a47c5d8f-3e2b-4f5a-9825-7ac8f11e2345",
  "messageTime": "2024-12-06T12:34:56Z",
  "messageVersion": "0.0.1",
  "subsystem": "ss0",
  "dataProvider": "spaceprotocol",
  "dataType": "ss0.data.eoobservation.calibrated",
  "dataVersion": "0.1.0",
  "traceability": {
    "internal": [
      {
        "dataType": "ss0.data.eoobservation",
        "messageId": "<parent_message_uuid>"
      }
    ],
    "external": [
      {
        "resourceLink":
            ↪ "https://api.spaceprotocol.org/sensor_calibration",
        "parameters": {
          "queryParameter": {
          "sensorid":"Panopticon_012",
          "timestamp":"2024-12-06T12:34:56Z"
        }
      }
      }
    ]
  }
}
```